

Bus seat booking

DDD · TDD · Clean Architecture

- ATUHAIRE MARY SEANICE · 2300706843 — Domain Lead; Slides 1–3
- kisakye rita · 2400700571 — Domain Model Lead; Slides 4–6
- Nassaka Catherine · 2400709816 — Architecture Lead; Slides 7–9
- Mwizerwa Timothy · 2400724330 — Testing Lead; Slides 11–12
- Mwizerwa Timothy also covers Integration Lead; Slides 10 and 13–15
- Makerere University · Advanced Programming group coursework

A seat must be sold once

Problem · scope · language

- Problem: reserve bus seats without duplicate bookings, and stop advertising a trip when its final seat is sold.
- Inside scope: schedule a trip, book a seat, retrieve a trip, list active advertisements, and withdraw an advertisement through an event.
- Outside scope: payments, cancellations, journey durations, overlapping journeys, web UI, database, and distributed messaging.
- Trip = one scheduled departure; Bus = vehicle identified by bus number; Booking = identified seat reservation.
- SeatNumber = positive integer value; TripAdvertisement = public availability record identified by trip number.
- Two connected use cases: BookSeat → WithdrawAdvertisement. Console inputs use local dates and times.

Exactly six business rules

Rule · responsible component · violation outcome

- BR1 — Value: a seat number must be a positive integer, excluding bool. SeatNumber rejects invalid values with ValueError.
- BR2 — Identity/state: booking an available seat adds an identified Booking to the same Trip and reduces availability. Trip.book_seat rejects empty passenger names or a full trip.
- BR3 — Invariant: each booked seat is unique and within 1..bus capacity. Trip.book_seat rejects duplicates/out-of-range seats; its collection is read-only to callers.
- BR4 — Cross-concept: the same bus cannot serve two trips at the same departure instant. BusSchedulingPolicy checks existing trips and rejects the proposed assignment before storage.
- BR5 — Follow-up: the final booking raises TripFullyBooked to request withdrawal of that trip's advertisement. TripAdvertisement permits active → withdrawn only; repeated withdrawal returns a rejection with no state change.
- BR6 — Lookup: booking requires an existing Trip; scheduling requires an unused trip number. Application services use ITripRepository and reject missing/duplicate identities before mutation.

Values and identities

BR1 and BR2

- `SeatNumber` has no separate identity: `SeatNumber(1)` equals another `SeatNumber(1)`. It is immutable and validates its value at construction.
- `Trip` is identified by `trip_number`, which remains stable as bookings change.
- `Booking` is a child entity identified by a generated `booking_id`; passenger name is data, not identity.
- `Bus` is identified operationally by `bus_number`; the scheduling policy compares that identity across trips.
- Entities explicitly compare and hash by their ID, even when state differs. `SeatNumber` retains value equality. T2 checks stable identity across booking states.
- Frozen fields protect identity; `Trip`'s private booking dictionary remains mutable through its methods. A read-only view protects callers.

Two roots protect two boundaries

BR3 · aggregate invariants

- Aggregate A root: Trip. Contents: Bus assignment and Booking children, each containing a SeatNumber.
- A invariant: no duplicate seat, every booked seat lies within capacity, and $\text{available_seats} = \text{capacity} - \text{booking count} \geq 0$.
- Trip creates bookings; callers receive a read-only view of the booking collection. Booking children have no independent repository.
- Aggregate B root: TripAdvertisement. Identity: trip_number. No child entities are needed for this small aggregate.
- B invariant: an active advertisement has positive availability; a withdrawn advertisement has zero availability. Withdrawal is one-way.
- B returns new immutable state from update_availability/withdraw. Only its own methods determine valid transitions.

A policy across trip assignments

BR4 · Domain Service · Factory decision

- BusSchedulingPolicy receives a proposed Bus, departure time, and existing Trip aggregates.
- It rejects equal departure times for an existing assignment with the same bus_number. Another bus at the same instant remains valid.
- This belongs in a Domain Service: the decision spans the proposed assignment and other Trip roots; no individual Trip owns the schedule.
- CreateTripService uses an indexed bus/time lookup, then invokes the injected SchedulingPolicy contract before storing the Trip and advertisement.
- No Factory: construction is small and constructors already validate inputs. A Factory would add indirection without a creation workflow.
- Deliberate limit: no journey-duration model, so this rule does not detect overlapping journeys at different departure times.

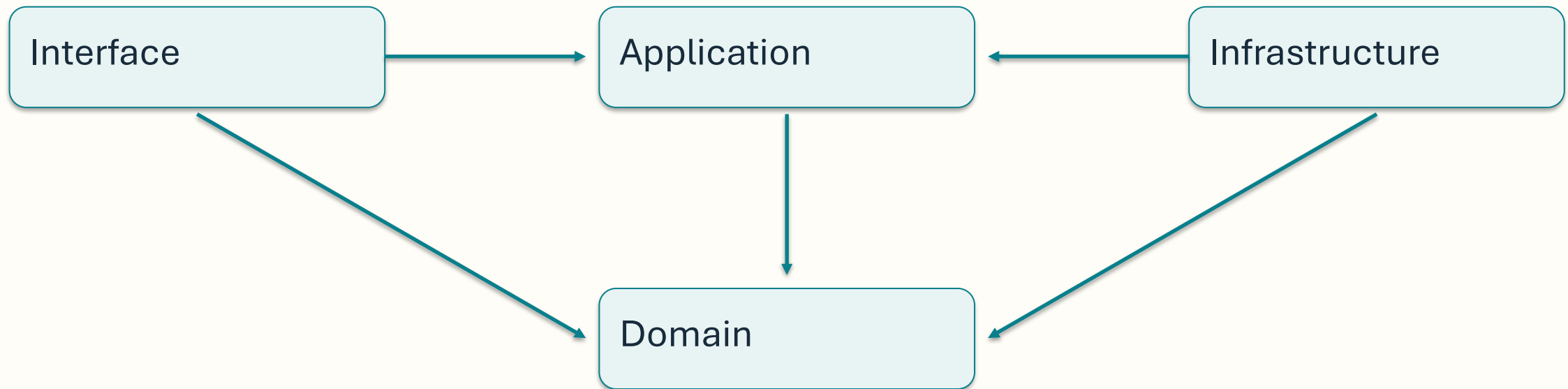
Repositories store aggregate roots

Contracts in Application · adapters in Infrastructure

- `ITripRepository` → `InMemoryTripRepository`: stores Trip roots keyed by `trip_number`, including their Booking children.
- Operations: `add`, `find_by_number`, `find_departures`, `list_all`, `save`. Reads and writes detach Trip state; changes are visible only after save.
- `ITripAdvertisementRepository` → `InMemoryTripAdvertisementRepository`: stores TripAdvertisement roots keyed by `trip_number`.
- Operations: `put`, `find_by_number`, `list_available`. Withdrawn roots are retained; available listings include active roots only.
- One abstraction per aggregate root used in the workflow. No separate Booking repository breaks the Trip boundary.
- Both adapters share `InMemoryStore` and an injected `UnitOfWork`. Its reentrant lock serializes workflows; exceptions restore both roots and the departure index.
- T6 checks the same contract with a separate scan-based reference adapter. Storage remains process-local; snapshots cost $O(n)$ and restart resets data.

Dependencies point toward policy

Static import arrows: caller → dependency



main.py composes all four areas; Domain imports only Domain and the standard library.

No Layer Supertype: no shared entity behaviour needs one. Repository ABCs are contracts.

BookSeatService coordinates

Input/output DTOs · dependency injection

- Input: BookSeatInputDTO(trip_number, passenger_name, seat_number). The console parses user input; the domain validates seat values.
- Service flow: repository lookup → SeatNumber → Trip.book_seat → save Trip → refresh/handle advertisement → output DTO.
- Output: booking details, available_seats, and AdvertisementOutcome with enum status and optional enum rejection reason.
- Business decisions stay in SeatNumber, Trip, TripAdvertisement, and BusSchedulingPolicy.
- main injects repositories, UnitOfWork, policy, updater, handler, and ConsoleView through contracts. Application services and the console controller depend on abstractions.
- The transaction covers lookup through follow-up. Storage exceptions roll back both roots. A business rejection is a returned outcome; its valid booking remains committed.

The final seat triggers withdrawal

BR5 · in-process aggregate communication

- Request → BookSeatService → Trip (A) → TripFullyBooked → TripFullyBookedHandler → TripAdvertisement (B)
- Before the event: a valid booking reduces available_seats to zero. Earlier bookings produce no event.
- Within one transaction, the application saves Trip and invokes the injected handler in-process. No broker is needed; nested calls share a reentrant lock.
- The handler retrieves B and calls withdraw(). B checks that it is active, then returns inactive state with zero availability.
- The handler stores the returned root; listings hide it. The DTO carries AdvertisementStatus.WITHDRAWN; the view formats it for users.
- Repeated delivery returns REJECTED / ALREADY_WITHDRAWN without state changes. Missing B returns REJECTED / MISSING; storage exceptions propagate and trigger rollback.

Eight tests, explicit coverage

test/test_architecture.py · expected results

- T1 / BR1: seat 1 accepted; zero, negative, bool, float, and string rejected.
- T2 / BR2: saved booking state, detached reads/writes, stable entity equality/hash, and read-only booking collection.
- T3 / BR3: capacity boundary, duplicate/range/name rejection, and concurrent same-seat booking with one winner.
- T4 / BR4: bus/time conflict, another bus accepted, concurrent scheduling with one winner and no orphan advertisement.
- T5 / BR5: final-seat event; substitute updater/handler implementations work through protocols without inheritance.
- T6 / BR6: lookup/duplicate rejection; shared contracts and complete booking flow run against two repository adapters.
- T7: concurrent distinct-seat bookings lose no updates; B withdraws once. Console controller/view flow also passes.
- T8: fresh persisted state after repeated rejection; missing B; rollback after Trip/B writes and scheduling failure. Eight scenarios, OK.

A real fail → implement → pass cycle

T2 isolation regression · evidence retained

- RED: added an assertion that mutating a retrieved Trip must leave stored state unchanged until save. The existing adapter failed: `AssertionError: 0 != 1`.
- Evidence: `evidence/isolation-red.txt` records the actual failing run before the fix.
- IMPLEMENT: detach Trip roots on add/read/save; inject a `UnitOfWork` to serialize workflows and roll back exceptions.
- GREEN: the T2 isolation assertion passes in the revised scenario. Evidence: `evidence/isolation-green.txt`; tests were expanded further after the first failure.
- REGRESSION: all eight scenarios pass, including concurrency, storage failures, adapter substitution, and controller/view behaviour.
- The earlier BR4 TDD cycle remains in `tdd-red.txt`, `tdd-green.txt`, and `tdd-change.diff`. Both cycles are real revision evidence, not claims about the original development history.

Walk through the main use case

TR001 · two seats · final booking

- Setup: schedule TR001 on BUS001 with capacity 2. The scheduling policy accepts; both Trip and its active advertisement are stored.
- Alex books seat 1. Trip availability becomes 1; no full-booking event. The advertisement updates to 1.
- Sam submits BookSeatInputDTO('TR001', 'Sam', 2). Lookup succeeds and SeatNumber(2) validates.
- Trip verifies range, capacity, uniqueness, and passenger name. It creates Booking, leaves zero seats, and raises TripFullyBooked('TR001').
- Within the UnitOfWork, application saves Trip. Handler asks B to withdraw; B accepts and returns its inactive state, which is stored. Commit releases the lock.
- Output reports the booking ID, zero remaining seats, and 'withdrawn'. Available-trip listing excludes TR001. T7 verifies this complete path.

Change a design when evidence demands it

Rejected choice · improved responsibility

- Rejected: the event handler directly deletes the advertisement through a repository. B then has no transition rule and cannot reject a follow-up.
- Chosen: retain B and let `TripAdvertisement.withdraw` guard the state change. The handler orchestrates and persists its result.
- Benefit: T8 can demonstrate a real domain rejection and unchanged final state. Retention also makes the outcome inspectable.
- Tradeoff: inactive records remain in memory; `list_available` filters them. Persistence is intentionally temporary.
- Change exercise: if repeated withdrawal should succeed idempotently, change B's transition contract and T8's expected outcome; booking rules need not change.
- MVC separation: `ConsoleApp` uses `ConsolePresenter`; `ConsoleView` implements it structurally. Decorators, callbacks, and immutable transitions serve concrete needs; no monad is needed here. AI help is disclosed in README.

Every rule has inspectable evidence

Rule → responsible code → test → final result

- BR1 → Domain/ValueObject/seatnumber.py : SeatNumber → T1 → PASS
- BR2 → Domain/Aggregate/TripAggregate/trip.py : book_seat → T2 → PASS
- BR3 → Trip.book_seat + booked_seats read-only view → T3 → PASS
- BR4 → Domain/Services/BusSchedulingPolicy.py : ensure_available → T4 → PASS; red/green logs retained
- BR5 → Trip + TripFullyBooked + handler + TripAdvertisement.withdraw → T5 / T7 / T8 → PASS
- BR6 → BookSeatService / CreateTripService + ITripRepository → T6 → PASS
- Evidence: evidence/test-results.txt; run `python -B -m unittest discover -s test -v` from the root.
- Conclusion: six rules, two roots, an event-connected workflow, eight tests, and 15 slides. Four confirmed members cover five roles; each member must defend the work individually.